

Practice Question Set

1. What four key properties does a storage class define for any variable in C?
2. What are the three variable scope levels described in C?
3. What is the default storage class for local variables declared inside a function or block?
4. Where are automatic (auto) variables allocated in memory, and what is their default initial value?
5. What is the purpose of declaring a variable with the register storage class keyword?
6. What critical limitation applies to variables declared with the register storage class?
7. Describe the key behavior of a static variable when used inside a function.
8. In which memory segment are static and extern variables stored?
9. What is the default initial value assigned by the system to static and extern variables if uninitialized?
10. What is the fundamental difference between declaring a variable using `extern int var;` versus defining it using `int var;`?

Important Interview/Viva Questions

11. Why does attempting to use the address-of operator (&) on a register variable result in a compilation error?
12. What fallback rule does the compiler apply if there are no free CPU registers available for a variable declared with register?

13. Explain how the initialization lifecycle of a static variable inside a function differs from an auto variable during repeated function calls.

14. Predict the output of the following code snippet and explain why the value changes:

```
#include <stdio.h>

void counter() {
    static int count = 0;
    count += 5;
    printf("%d ", count);
}

int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

15. How does the extern keyword enable variable sharing across multiple C source files?

16. Compare auto, register, static, and extern variables in terms of their Storage Location, Default Value, Scope, and Lifetime.

Answers

1. What four key properties does a storage class define for any variable in C?

Answer:

A storage class defines:

Storage Location: Where memory is reserved (Stack, CPU Registers, or Data Segment).

Default Initial Value: System value assigned if none is provided explicitly (e.g., garbage vs. zero).

Scope (Visibility): Region of code where the variable can be directly accessed.

Lifetime (Longevity): How long the variable retains its memory space during program execution.

2. What are the three variable scope levels described in C?

Answer:

Block Scope: Accessible only within the { ... } block where declared.

Function Scope: Accessible throughout the enclosing function.

Program (Global) Scope: Accessible across all functions and files within the program.

3. What is the default storage class for local variables declared inside a function or block?

Answer:

The **Automatic (auto)** storage class is the default for local variables. Writing `auto int x;` is identical to writing `int x;`

4. Where are automatic (auto) variables allocated in memory, and what is their default initial value?

Answer:

Storage Location: Stack Segment.

Default Initial Value: Garbage Value (unpredictable).

5. What is the purpose of declaring a variable with the register storage class keyword?

Answer:

It hints to the compiler to store the variable directly inside a CPU Register instead of RAM (Stack) to dramatically cut access times for frequently referenced variables, such as loop counters.

6. What critical limitation applies to variables declared with the register storage class?

Answer:

You cannot use the address-of operator (&) on a register variable because CPU registers do not have memory addresses in RAM.

7. Describe the key behavior of a static variable when used inside a function.

Answer:

A static variable preserves its stored value across multiple function calls. It is initialized only once when execution reaches its definition, and subsequent call cycles skip re-initialization.

8. In which memory segment are static and extern variables stored?

Answer:

Both static and extern variables are allocated in the **Data Segment**.

9. What is the default initial value assigned by the system to static and extern variables if uninitialized?

Answer:

Both static and extern variables default automatically to **Zero (0)**.

10. What is the fundamental difference between declaring a variable using extern int var; versus defining it using int var;?

Answer:

int var; is a **definition**, which allocates new memory space for the variable.

extern int var; is a **declaration**, which informs the compiler that the variable exists elsewhere without allocating new memory.

11. Why does attempting to use the address-of operator (&) on a register variable result in a compilation error?

Answer:

The address-of operator (&) queries memory locations in RAM (Stack). Because register variables are requested to be stored directly inside CPU Registers rather than RAM, they do not possess a RAM memory address, causing the compiler to throw an error (int *ptr = &i; is invalid).

12. What fallback rule does the compiler apply if there are no free CPU registers available for a variable declared with register?

Answer:

If no CPU registers are free, the compiler automatically falls back to treating the variable as a standard automatic (auto) local variable allocated on the Stack.

13. Explain how the initialization lifecycle of a static variable inside a function differs from an auto variable during repeated function calls.

Answer:

auto variable: Allocated on the Stack and re-initialized every time the function is called. Its memory is destroyed as soon as the execution leaves the function block.

static variable: Allocated in the Data Segment and initialized **only once** at program startup. When the function terminates, its memory remains intact, allowing its modified value to persist for subsequent function invocations.

14. Predict the output of the code snippet and explain why the value changes.

Answer:

Output: 5 10 15

Explanation: count is initialized to 0 only once on program startup.

Call 1: count becomes $0 + 5 = 5$ and prints 5.

Call 2: count reuses the persisted value 5, becomes $5 + 5 = 10$, and prints 10.

Call 3: count reuses 10, becomes $10 + 5 = 15$, and prints 15.

15. How does the extern keyword enable variable sharing across multiple C source files?

Answer:

The global variable is defined (memory allocated) in one main source file (e.g., `int x = 10; in main.c`). In secondary files (e.g., `support.c`), declaring `extern int x;` informs the compiler that `x` is defined externally. This allows functions in `support.c` to read and modify the shared variable `x` without creating duplicate memory allocations.

16. Compare auto, register, static, and extern variables in terms of their Storage Location, Default Value, Scope, and Lifetime.

Answer:

Storage Class	Keyword	Default Value	Storage Location	Scope	Lifetime
---------------	---------	---------------	------------------	-------	----------

Storage Class	Keyword	Default Value	Storage Location	Scope	Lifetime
Automatic	auto	Garbage Value	Stack Segment	Block / Local	Until end of current block
Register	register	Garbage Value	CPU Register	Block / Local	Until end of current block
Static	static	Zero (0)	Data Segment	Block / Local	Until program terminates
External	extern	Zero (0)	Data Segment	Progra	